# A Beginner-Friendly Guide to `main.py`

## The big idea

This program teaches a computer to label every tiny square in a picture. A tiny square is a pixel. The possible labels are background, red, green, and blue. It first makes practice pictures where it already knows the answers. Then it makes small corrections after mistakes. Finally, it labels a real picture. You do not need to know CNNs: here, a network is just a collection of adjustable numbers. Each code block appears before its simple explanation.

## 1. Tools and settings

```
"""Train a tiny, NumPy-only semantic-segmentation classifier.

The program makes synthetic 32x32 RGB images containing red, green, and blue
circles on varied textured backgrounds.  It learns to label every pixel as
background/red/green/blue using the pixel's 5x5 local neighbourhood, then
applies that learned model to ``test2.jpg`` and saves a mask plus an overlay.
"""

from PIL import Image  # Pillow reads the test image and writes PNG results.
import numpy as np      # NumPy supplies array operations and the neural network.



# A fixed seed makes the generated training set and training run reproducible.
rng = np.random.default_rng(42)

IMAGE_SIZE = 32          # Synthetic training images are IMAGE_SIZE x IMAGE_SIZE.
NUMBER_OF_IMAGES = 150   # Total generated images; each contributes 1,024 pixels.
PATCH_SIZE = 5           # Odd size keeps the current pixel centered in its patch.

HIDDEN_SIZE = 32         # Number of ReLU units in the single hidden layer.
OUTPUT_SIZE = 4          # Classes: background, red, green, and blue.

EPOCHS = 1000             # Number of balanced gradient-descent updates.
LEARNING_RATE = 0.05      # Step size for each parameter update.
SAMPLES_PER_CLASS = 256   # Per-class samples in each training batch.

TEST_IMAGE_PATH = "test2.jpg"  # Image classified after training completes.
```

**Simple explanation.** Pillow opens and saves images. NumPy stores large grids of numbers. The seed 42 makes random choices repeatable. The settings choose picture size, how much nearby area to look at, how many practice rounds to use, and how large each correction should be.

## 2. Background colors

```
# ------------------------------------------------
# Background colors
# ------------------------------------------------

# These colors should be classified as background.
BACKGROUND_COLORS = np.array([
    [0.02, 0.02, 0.02],  # black
    [0.20, 0.20, 0.20],  # dark gray
    [0.50, 0.50, 0.50],  # gray
    [0.90, 0.90, 0.90],  # white
    [0.78, 0.55, 0.42],  # light skin
    [0.48, 0.30, 0.20],  # dark skin/brown
    [0.38, 0.19, 0.08],  # brown
    [0.72, 0.62, 0.18],  # yellow/gold
    [0.42, 0.20, 0.48],  # muted purple
    [0.18, 0.42, 0.46]   # muted cyan
], dtype=np.float32)
```

**Simple explanation.** These are examples of colors that should be called background. They include dark, light, brown, yellow, purple, and cyan. This teaches the computer that background does not always have one appearance. Each row holds red, green, and blue amounts from 0 to 1.

## 3. Make one practice picture

```
# Generate one synthetic training image


def make_training_image():
    """Return one synthetic RGB image and its per-pixel integer class mask.

    The image starts as a randomly selected, noisy background.  Isolated
    coloured pixels are deliberately left unlabeled as background, while one
    to three filled coloured discs are assigned labels 1-3.  This distinction
    teaches the model that local area/context matters, not just one RGB value.
    """
    # Choose a base color, then vary its brightness so background appearance
    # is diverse rather than tied to a single intensity.
    background_index = int(
        rng.integers(0, len(BACKGROUND_COLORS))
    )

    background_color = BACKGROUND_COLORS[background_index]

    # Background brightness can be dark or bright.
    background_scale = rng.uniform(0.15, 1.0)

    image = np.ones(
```

```
        (IMAGE_SIZE, IMAGE_SIZE, 3),
        dtype=np.float32
    )

    image *= background_color * background_scale

    # Add background texture.
    image += rng.normal(
        0.0,
        0.025,
        size=image.shape
    ).astype(np.float32)

    image = np.clip(image, 0.0, 1.0)

    # 0 = background
    # 1 = red
    # 2 = green
    # 3 = blue
    mask = np.zeros(
        (IMAGE_SIZE, IMAGE_SIZE),
        dtype=np.int64
    )

    # Add isolated colored noise pixels.
    # They remain background because they do not form an area.
    number_of_noise_pixels = 20

    for _ in range(number_of_noise_pixels):
        noise_y = int(rng.integers(0, IMAGE_SIZE))
        noise_x = int(rng.integers(0, IMAGE_SIZE))
        noise_class = int(rng.integers(1, 4))

        noise_strength = rng.uniform(0.02, 1.0)

        noise_color = rng.uniform(
            noise_strength * 0.05,
            noise_strength * 0.70,
            size=3
        )

        noise_color[noise_class - 1] = noise_strength

        image[noise_y, noise_x] = noise_color

    # Open grids broadcast efficiently to form a circle equation for all pixels
        .
    yy, xx = np.ogrid[:IMAGE_SIZE, :IMAGE_SIZE]

    # Add one to three colored areas.
    number_of_areas = int(rng.integers(1, 4))

    for _ in range(number_of_areas):
        radius = int(rng.integers(3, 8))

        center_y = int(
```

```
            rng.integers(radius, IMAGE_SIZE - radius)
        )

        center_x = int(
            rng.integers(radius, IMAGE_SIZE - radius)
        )

        area = (
            (yy - center_y) ** 2
            + (xx - center_x) ** 2
            <= radius ** 2
        )

        color_class = int(rng.integers(1, 4))

        # All classes include very dark and bright examples.
        strength = rng.uniform(0.02, 1.0)

        # Other channels may be as high as 75% of
        # the dominant channel, allowing muted colors.
        color = rng.uniform(
            strength * 0.05,
            strength * 0.75,
            size=3
        )

        color[color_class - 1] = strength

        # Texture amount scales with brightness.
        texture_amount = max(
            0.002,
            strength * 0.06
        )

        area_texture = rng.normal(
            0.0,
            texture_amount,
            size=(np.sum(area), 3)
        )

        image[area] = np.clip(
            color + area_texture,
            0.0,
            1.0
        )

        mask[area] = color_class

    return image, mask
```

**Simple explanation.** This function makes one picture and its correct answer sheet, called a mask. The picture starts as a noisy background, which has answer 0. Single colored dots are still called background; this teaches that one colored pixel alone is not a real colored object. Then the code draws colored circles: red pixels get answer 1, green get 2, and blue get 3. The function returns both the picture and its known answers.

## 4. Turn pixels into clues

```
# Convert images into 5 x 5 patch features

def extract_features(rgb_images):
    """Convert a batch ``(N, H, W, 3)`` of RGB images into flat patch vectors.

    Each pixel gets normalized R/G/B (chromatic ratios) plus maximum-channel
    brightness.  Reflection padding lets border pixels receive a full 5x5
    neighbourhood.  The returned shape is ``(N*H*W, PATCH_SIZE*PATCH_SIZE*4)``;
    rows align exactly with ``masks.reshape(-1)`` labels elsewhere in the file.
    """
    number_of_images = rgb_images.shape[0]
    height = rgb_images.shape[1]
    width = rgb_images.shape[2]

    # Separate color from intensity: dividing by maximum brightness makes a
    # dark red and bright red similar in chromaticity while retaining
       brightness.
    brightness = np.max(
        rgb_images,
        axis=3,
        keepdims=True
    )

    normalized_rgb = rgb_images / np.maximum(
        brightness,
        1e-6
    )

    # Four values per pixel:
    # normalized R, G, B, and brightness.
    network_images = np.concatenate(
        [normalized_rgb, brightness],
        axis=3
    )

    padding = PATCH_SIZE // 2

    padded_images = np.pad(
        network_images,
        (
            (0, 0),
            (padding, padding),
            (padding, padding),
            (0, 0)
        ),
        mode="reflect"
    )

    # This creates a patch view without manually looping over every pixel.
    patches = np.lib.stride_tricks.sliding_window_view(
        padded_images,
        (PATCH_SIZE, PATCH_SIZE),
        axis=(1, 2)
    )
```

```
54
55      features = patches.reshape(
56          number_of_images * height * width,
57          PATCH_SIZE * PATCH_SIZE * 4
58      )
59
60      return features
```

**Simple explanation.** The computer needs number clues instead of descriptions like "red area." It keeps red, green, blue, and brightness clues for every pixel. It also looks at a 5 by 5 area around that pixel. This helps it tell a true colored area from a lone dot. The result is 100 clues per pixel: 25 nearby pixels times 4 clues.

## 5. Build the practice set

```
1  # Generate training data
2
3
4  # Keep images and masks as lists while generating, then stack them into arrays.
5  images = []
6  masks = []
7
8  for _ in range(NUMBER_OF_IMAGES):
9      training_image, training_mask = make_training_image()
10
11     images.append(training_image)
12     masks.append(training_mask)
13
14 images = np.array(images, dtype=np.float32)
15 masks = np.array(masks, dtype=np.int64)
16
17 # X has one feature row per pixel; flattening masks uses the identical ordering
       .
18 X = extract_features(images)
19 y = masks.reshape(-1)
20
21 print("Images:", images.shape)
22 print("Masks:", masks.shape)
23 print("Network␣inputs:", X.shape)
24 print("Network␣labels:", y.shape)
```

**Simple explanation.** The loop makes 150 practice pictures. `X` contains the clues and `y` contains the matching correct answers. The printed sizes check that clues and answers match before learning begins.

## 6. Start the adjustable numbers

```
1  # Initialize neural network
2
3
4  # The feature width is derived rather than hard-coded: 5*5 pixels * 4 values =
       100.
5  INPUT_SIZE = X.shape[1]
6
```

```
# He initialization scales random weights for stable variance through a ReLU
    layer.
weights_1 = rng.normal(
    0,
    np.sqrt(2.0 / INPUT_SIZE),
    size=(INPUT_SIZE, HIDDEN_SIZE)
).astype(np.float32)

bias_1 = np.zeros(
    HIDDEN_SIZE,
    dtype=np.float32
)

weights_2 = rng.normal(
    0,
    np.sqrt(2.0 / HIDDEN_SIZE),
    size=(HIDDEN_SIZE, OUTPUT_SIZE)
).astype(np.float32)

bias_2 = np.zeros(
    OUTPUT_SIZE,
    dtype=np.float32
)

class_names = [
    "background",
    "red",
    "green",
    "blue"
]
```

**Simple explanation.** The network has adjustable numbers called weights and biases. Think of a weight as how much attention it gives a clue. They start random because nothing has been learned yet. The first group turns 100 clues into 32 helper values. The second group makes four scores, one for every answer choice.

## 7. Give every color fair practice

```
# Find training positions for every class


# Count and record all available rows per label so every batch can be balanced.
class_counts = np.bincount(
    y,
    minlength=OUTPUT_SIZE
)

class_indices = []

for class_index, class_name in enumerate(class_names):
    print(
        class_name,
        "pixels:",
        class_counts[class_index]
    )
```

```
18
19      indices = np.where(y == class_index)[0]
20      class_indices.append(indices)
```

**Simple explanation.** There are many more background pixels than colored pixels. Without this block, the computer could guess background almost all the time. This saves the locations of every class so practice can use equal amounts of background, red, green, and blue.

## 8. Make guesses during practice

```
1   # Training loop
2
3
4   for epoch in range(EPOCHS):
5       batch_indices = []
6
7       # Build a balanced batch.  Background is much more common than colored
8       # pixels, so sampling the same count for each class prevents its dominance.
9       for class_index in range(OUTPUT_SIZE):
10          selected = rng.choice(
11              class_indices[class_index],
12              size=SAMPLES_PER_CLASS,
13              replace=False
14          )
15
16          batch_indices.append(selected)
17
18      batch_indices = np.concatenate(batch_indices)
19      rng.shuffle(batch_indices)
20
21      X_batch = X[batch_indices]
22      y_batch = y[batch_indices]
23
24      batch_size = len(y_batch)
25
26      # Forward pass, layer 1: affine transformation then ReLU non-linearity.
27      hidden_scores = X_batch @ weights_1 + bias_1
28      hidden_output = np.maximum(0, hidden_scores)
29
30      # Forward pass, layer 2: produces unnormalized logits, one per class.
31      scores = hidden_output @ weights_2 + bias_2
32
33      # Softmax converts logits into class probabilities.  Subtracting each row's
34      # maximum does not change probabilities but avoids exponential overflow.
35      shifted_scores = scores - np.max(
36          scores,
37          axis=1,
38          keepdims=True
39      )
40
41      exp_scores = np.exp(shifted_scores)
42
43      probabilities = exp_scores / np.sum(
44          exp_scores,
45          axis=1,
```

```
        keepdims=True
    )
```

**Simple explanation.** One epoch is one practice round. The code chooses equal examples from every class, mixes them, and asks for guesses. The first layer makes helper values. Negative helper values become zero; ReLU is just the name for this "ignore unhelpful signal" step. The second layer makes four scores. Softmax changes them into four percentages that add up to 1.

## 9. Learn from mistakes

```
    # Cross-entropy penalizes low probability on the known correct class;
    # accuracy is a more intuitive diagnostic but is not optimized directly.
    correct_probabilities = probabilities[
        np.arange(batch_size),
        y_batch
    ]

    loss = -np.mean(
        np.log(correct_probabilities + 1e-12)
    )

    predictions = np.argmax(
        probabilities,
        axis=1
    )

    accuracy = np.mean(
        predictions == y_batch
    )

    # For softmax + cross-entropy, dL/dlogits is probability minus one-hot
       label.
    score_gradient = probabilities.copy()

    score_gradient[
        np.arange(batch_size),
        y_batch
    ] -= 1

    score_gradient /= batch_size

    # Backpropagate through the output affine layer to its parameters.
    weights_2_gradient = (
        hidden_output.T @ score_gradient
    )

    bias_2_gradient = np.sum(
        score_gradient,
        axis=0
    )

    # Backpropagate through layer 1.  ReLU has zero derivative for nonpositive
    # pre-activation values, so those units contribute no gradient.
    hidden_gradient = (
        score_gradient @ weights_2.T
```

```
    )

    hidden_gradient[hidden_scores <= 0] = 0

    weights_1_gradient = (
        X_batch.T @ hidden_gradient
    )

    bias_1_gradient = np.sum(
        hidden_gradient,
        axis=0
    )

    # Gradient descent moves parameters opposite each loss gradient.
    weights_1 -= (
        LEARNING_RATE * weights_1_gradient
    )

    bias_1 -= (
        LEARNING_RATE * bias_1_gradient
    )

    weights_2 -= (
        LEARNING_RATE * weights_2_gradient
    )

    bias_2 -= (
        LEARNING_RATE * bias_2_gradient
    )

    if epoch % 50 == 0 or epoch == EPOCHS - 1:
        print(
            "Epoch:", epoch,
            "Loss:", loss,
            "Accuracy:", accuracy
        )
```

**Simple explanation.** Loss is one number saying how wrong the guesses were: lower is better. Accuracy is how often the direct guess was right. Backpropagation is like reviewing a wrong quiz answer: the program works backward to find which internal numbers pointed it the wrong way, then corrects those numbers a little. It prints progress every 50 rounds.

## 10. Save what was learned

```
# Save trained model


np.savez(
    "segmentation_model.npz",
    weights_1=weights_1,
    bias_1=bias_1,
    weights_2=weights_2,
    bias_2=bias_2,
    patch_size=PATCH_SIZE
)
```

```
# The .npz archive preserves learned parameters and the patch size for reuse.
```

**Simple explanation.** The learned numbers are saved in `segmentation_model.npz`. This is the trained model and can be used later without repeating all practice.

## 11. Open the real picture

```
# Load downloaded image


# Convert to RGB so grayscale/palette/JPEG input all have the expected 3
    channels.
picture = Image.open(
    TEST_IMAGE_PATH
).convert("RGB")

test_image = (
    np.asarray(picture, dtype=np.float32) / 255.0
)

height, width, _ = test_image.shape

print("Downloaded image:", test_image.shape)
```

**Simple explanation.** The code opens `test2.jpg`, makes sure it has normal red-green-blue colors, and changes color values from 0–255 to 0–1. This makes it match the practice pictures. It also remembers the image size.

## 12. Label every pixel

```
# Predict the downloaded image


# Adding a leading dimension treats the single test image as a batch of one.
test_X = extract_features(
    test_image[None, :, :, :]
)

test_hidden_scores = (
    test_X @ weights_1 + bias_1
)

test_hidden_output = np.maximum(
    0,
    test_hidden_scores
)

test_scores = (
    test_hidden_output @ weights_2 + bias_2
)

# The most likely class for each flattened pixel is reshaped back to image
    layout.
predicted_mask = np.argmax(
```

```
    test_scores,
    axis=1
).reshape(height, width)
```

**Simple explanation.** The real picture gets the same clues as the practice pictures. The trained numbers make four scores for every pixel. The code chooses the largest score, then rebuilds the long answer list into a grid shaped like the original image.

## 13. Check obvious colors

```
# Diagnose strongest red, green, and blue pixels

color_information = [
    ("red", 0, 1),
    ("green", 1, 2),
    ("blue", 2, 3)
]

for color_name, channel_index, expected_class in color_information:
    # This diagnostic picks the pixel where the target channel exceeds the
    # strongest competing channel by the largest amount, then reports its model
    # probabilities.  It does not affect the predicted mask or training.
    other_channels = [
        index
        for index in range(3)
        if index != channel_index
    ]

    target_channel = test_image[:, :, channel_index]

    strongest_other = np.max(
        test_image[:, :, other_channels],
        axis=2
    )

    color_strength = target_channel - strongest_other

    pixel_y, pixel_x = np.unravel_index(
        np.argmax(color_strength),
        color_strength.shape
    )

    pixel_index = pixel_y * width + pixel_x
    pixel_scores = test_scores[pixel_index]

    shifted = pixel_scores - np.max(pixel_scores)
    pixel_probabilities = np.exp(shifted)
    pixel_probabilities /= np.sum(pixel_probabilities)

    predicted_class = int(
        np.argmax(pixel_probabilities)
    )

    print()
    print("Target␣color:", color_name)
```

```
    print("Location:", pixel_x, pixel_y)

    print(
        "RGB:",
        test_image[pixel_y, pixel_x] * 255
    )

    print(
        "Brightness:",
        np.max(test_image[pixel_y, pixel_x])
    )

    print("Probabilities:", pixel_probabilities)
    print("Expected:", class_names[expected_class])
    print("Predicted:", class_names[predicted_class])
```

**Simple explanation.** This is a check for you; it does not alter the final answer. For red, green, and blue, it finds a pixel where that color is especially strong and prints its location, true color, brightness, and the computer's percentages.

## 14. Draw and save the answer

```
# Create mask and overlay


# Map integer classes to display colors for an easily inspected PNG mask.
mask_colors = np.array([
    [0, 0, 0],
    [255, 0, 0],
    [0, 255, 0],
    [0, 0, 255]
], dtype=np.uint8)

original = np.clip(
    test_image * 255,
    0,
    255
).astype(np.uint8)

colored_mask = mask_colors[predicted_mask]

overlay = original.copy()
detected_pixels = predicted_mask != 0

# Only non-background predictions are tinted; the original remains visible
    below.
overlay[detected_pixels] = (
    0.35 * original[detected_pixels]
    + 0.65 * colored_mask[detected_pixels]
).astype(np.uint8)

Image.fromarray(colored_mask).save(
    "downloaded_predicted_mask.png"
)

```

```
Image.fromarray(overlay).save(
    "downloaded_overlay.png"
)

print()
print("Background pixels:", np.sum(predicted_mask == 0))
print("Red pixels:", np.sum(predicted_mask == 1))
print("Green pixels:", np.sum(predicted_mask == 2))
print("Blue pixels:", np.sum(predicted_mask == 3))

print("Saved segmentation_model.npz")
print("Saved downloaded_predicted_mask.png")
print("Saved downloaded_overlay.png")
```

**Simple explanation.** The numeric answers become visible colors: black for background, then red, green, and blue. This saves a color-only mask. The code also makes an overlay that keeps the original picture visible but adds a strong color tint where a colored object was detected. Finally it saves both pictures and prints pixel counts.

## One-sentence summary

The program practices on pictures where it knows the answers, gently corrects itself after mistakes, and uses what it learned to color every pixel of a new picture.